

Warp-Level Programming: Placement and Interview Guide

This guide uses NVIDIA CUDA terminology. A **warp** is normally 32 threads (`warpSize == 32`) that execute in SIMT fashion. Warp-level code is fast because lanes communicate through registers and warp masks, but it is correct only when participation and synchronization rules are respected.

Warp Shuffle Operations

1. Overview

Warp shuffle operations let one thread, called a **lane**, read a register value held by another lane in the same warp. The value moves through an on-chip warp data path rather than through shared or global memory. CUDA exposes direct, up, down, and XOR shuffle patterns.

They matter because register exchange is usually lower-latency and needs fewer instructions than the traditional “store to shared memory, synchronize, load” sequence. Shuffles are used in reductions, scans, broadcasts, sorting networks, matrix kernels, and voting/compaction code. Interviewers ask about them to test whether a candidate understands warps, lane IDs, masks, and the limits of warp-synchronous programming.

2. Core Idea

Imagine 32 people each holding a card. A shuffle is an instruction saying, “without passing your own card away, copy the card held by person k .” Every participant can ask for a different source in the same instruction.

```
int lane = threadIdx.x & 31;
int x = lane * 10;
int from_lane_0 = __shfl_sync(0xffffffffu, x, 0);
// Every lane now has 0 because lane 0 held 0.
```

Step by step:

1. Each lane computes x in a register.
2. All lanes named by the mask execute the shuffle.
3. The hardware routes lane 0's x to every participating destination lane.
4. The original x in each lane remains unchanged; the returned value is a copy.

The operation is warp-local. It cannot read a register from another warp, and reading from an inactive or nonparticipating source lane gives an undefined result.

3. Important Subtopics

Subtopic	What it means and why it matters	Example	Common interview angle
Direct shuffle	Read from an explicitly numbered source lane.	<code>__shfl_sync(mask, x, 0)</code> broadcasts lane 0.	Best choice for broadcast or indexed exchange.
Shuffle down	Read from <code>lane + delta</code> .	<code>__shfl_down_sync(mask, x, 16)</code> starts a reduction.	Explain why upper lanes may receive an unchanged/unspecified partner value and should not contribute it blindly.
Shuffle up	Read from <code>lane - delta</code> .	Inclusive scan with deltas 1, 2, 4, 8, 16.	Guard with <code>lane >= delta</code> .
XOR shuffle	Read from <code>lane ^ laneMask</code> .	Butterfly exchange for all-reduce.	Useful for power-of-two groups and symmetric communication.
Participation mask	Names lanes required to rendezvous at the intrinsic.	Mask returned by <code>__ballot_sync</code> .	A mask is a contract, not merely an output filter.
Width	Divides a warp into logical power-of-two subgroups.	<code>width=8</code> creates four logical groups.	Width does not make arbitrary sparse masks contiguous.
Data types	Modern CUDA overloads support common 32/64-bit scalar types; values can also be packed when appropriate.	Shuffle an <code>int</code> , <code>float</code> , or supported 64-bit value.	Do not claim arbitrary structures are automatically supported.

4. Real-World Example

In a softmax kernel, each lane loads several logits and computes a local maximum. Warp shuffles then reduce those local maxima to one warp maximum. After exponentiation, another shuffle reduction computes the sum. This avoids two shared-memory reductions and their block-wide barriers. Larger softmax rows still need a second stage across warps, often using shared memory or a block cooperative group.

5. Diagrams / Mental Models

```
Initial values: lane 0  lane 1  lane 2  lane 3
                [ 8 ]  [ 3 ]  [ 5 ]  [ 1 ]

shuffle down by 1 returns:
                [ 3 ]  [ 5 ]  [ 1 ]  [ out-of-range ]

shuffle XOR 1 returns:
```

```
[ 3 ] [ 8 ] [ 1 ] [ 5 ]
0<->1      2<->3
```

Mental model: **shuffle copies a register value across lanes; it does not move thread execution or create cross-warp communication.**

6. Common Interview Questions

1. **What is a warp shuffle?** It is a warp-level register exchange in which a lane reads a value from another lane. Expected: same warp, no shared-memory round trip. Mistake: saying registers become shared globally.
2. **Why can shuffle be faster than shared memory?** It removes explicit stores, loads, address calculation, and often a block barrier. Expected: lower instruction and synchronization overhead. Mistake: promising a fixed speedup on every GPU.
3. **What shuffle variants exist?** Direct, up, down, and XOR. Expected: know broadcast, scan, reduction, and butterfly uses. Mistake: treating them as interchangeable without lane guards.
4. **Can a shuffle communicate across warps?** No. Expected: use shared memory, global memory, or a larger cooperative group for wider scope. Mistake: using a block-wide mask.
5. **Does shuffle destroy the source value?** No; it returns a copy. Mistake: describing it like a destructive move.
6. **What does width do?** It partitions a warp into logical power-of-two segments and wraps/limits lane addressing according to the intrinsic. Expected: subgroup interpretation. Mistake: assuming it selects active lanes.
7. **What if the source lane is inactive?** The returned value is undefined. Expected: validate participation and source existence. Mistake: assuming zero is returned.
8. **Why do _sync intrinsics take a mask?** The mask specifies participating lanes that must rendezvous and enables correct use after divergence. Mistake: calling it only a performance hint.
9. **When should shared memory still be used?** For cross-warp exchange, reusable arrays, arbitrary indexing, or block-level collectives. Mistake: claiming shuffle replaces shared memory.
10. **What is a common partial-warp bug?** Using `0xffffffff` when the last warp has fewer valid threads, or reading a source absent from the valid mask. Expected: derive a valid mask and guard partners. Mistake: assuming masked shuffle supplies an identity value.

7. Deep-Dive Questions

1. **Why is a shuffle not a general memory fence?** It synchronizes participating lanes for the intrinsic and transfers its register operand; it does not publish arbitrary shared/global writes. Use the synchronization primitive whose documented memory ordering matches the data.
2. **How does XOR shuffle implement an all-reduce?** At offsets 16, 8, 4, 2, 1, each lane exchanges with a partner differing in one lane-ID bit and combines values. After all stages, every lane has the result for a full power-of-two group.
3. **How do logical widths affect source lanes?** Lane addressing is interpreted within each aligned subgroup of `width`; `width` must be a power of two no larger than `warpSize`.
4. **Can divergent branches use shuffles safely?** Yes, if every non-exited lane named in the mask reaches the same intrinsic with that mask and every requested source is participating. Compute the membership mask before or with a correct vote.

5. **Why might shuffle code not improve performance?** The kernel may be limited by global memory, register pressure may reduce occupancy, or the compiler may already optimize the shared-memory pattern. Measure rather than assuming.

8. Comparison Tables

Mechanism	Scope	Storage path	Synchronization	Best use
Warp shuffle	One warp	Register-to-register	Named lanes rendezvous	Reduction, scan, broadcast
Shared memory	One block	On-chip shared array	Usually <code>__syncthreads()</code>	Cross-warp and reusable exchange
Global memory	Grid/device	Device memory/cache	Separate ordering/sync needed	Persistent or grid-wide exchange

Variant	Source lane pattern	Typical algorithm
<code>__shfl_sync</code>	Explicit lane	Broadcast/gather
<code>__shfl_up_sync</code>	<code>lane - delta</code>	Prefix scan
<code>__shfl_down_sync</code>	<code>lane + delta</code>	Tree reduction
<code>__shfl_xor_sync</code>	<code>lane ^ mask</code>	Butterfly all-reduce

9. Common Mistakes

- Using a full-warp mask inside a branch taken by only some lanes.
- Assuming a nonparticipating source returns zero.
- Forgetting a boundary guard in scans or partial reductions.
- Treating `width` as a substitute for the participation mask.
- Trying to exchange values between warps.
- Assuming shuffle synchronizes unrelated memory operations.
- Optimizing a simple shared-memory implementation without profiling.

10. Edge Cases / Special Cases

- The final warp of a block or grid may contain threads that exist physically but are logically out of range.
- A sparse mask such as lanes `{0, 2, 5}` is not made into a packed three-lane group by setting `width`.
- An out-of-range source behavior depends on the shuffle variant; code should guard use of values whose logical partner does not exist.
- A thread that has exited cannot participate even if its bit appears in an old mask.
- CUDA defines `warpSize`; hard-coded 32 is common for NVIDIA-specific code but `warpSize` better expresses intent.

11. How to Explain in Interview

“Warp shuffles let a CUDA lane directly read a register value from another lane in the same warp. They are ideal for broadcasts, reductions, and scans because they avoid shared-memory traffic and block-wide barriers.

Correctness depends on passing the right participation mask and never reading from a source lane that is inactive.”

12. Quick Revision Notes

- Definition: register exchange among lanes of one warp.
- Variants: direct, up, down, XOR.
- Strength: low-overhead intra-warp communication.
- Limit: no cross-warp communication.
- Must remember: mask = lanes required to participate; source must be active.
- Trap: `width` partitions lane numbering but does not repair an arbitrary sparse mask.

13. Practice Tasks

1. Broadcast lane 3's integer to an entire warp.
2. Implement an inclusive scan with `__shfl_up_sync` and trace eight lanes on paper.
3. Implement a maximum reduction with `__shfl_down_sync`.
4. Split a warp into four groups of eight using `width=8` and test a broadcast per group.
5. Modify a kernel so its last partial warp is correct.
6. Compare a shared-memory warp reduction against a shuffle version with Nsight Compute.

14. Final Cheat Sheet

Item	Answer
Core definition	Same-warp register exchange
Why it matters	Fewer memory operations and barriers
Most asked	Variants, masks, partial warps, shuffle vs shared memory
Key comparison	Shuffle is warp-local; shared memory supports the block
Interview trap	Inactive source lane gives undefined data, not an identity
One-line answer	“Shuffle is fast register communication among participating lanes of one warp.”

`__shfl_sync`

1. Overview

`__shfl_sync(mask, value, srcLane, width)` is the direct CUDA shuffle intrinsic. Each calling lane receives `value` from `srcLane`, interpreted within its logical subgroup. The default `width` is `warpSize`.

It matters because one instruction expresses broadcast or indexed register exchange. It appears in warp broadcasts, pivot selection, lookup exchange, reductions with explicit indices, and algorithms where every lane chooses a source. Interviewers use it to probe the exact meaning of `mask`, `srcLane`, and `width`, not merely syntax recall.

2. Core Idea

```

unsigned mask = __activemask();
int lane = threadIdx.x % warpSize;
float x = input[threadIdx.x];
float leader_value = __shfl_sync(mask, x, 0);

```

For a full warp, lane 0 supplies its `x`; all other lanes named in `mask` copy it. The mask specifies which non-exited lanes must execute this same shuffle. `srcLane=0` is relative to the logical subgroup selected by `width`.

Analogy: every table of 32 students has numbered seats. The teacher says, "all listed students copy the answer from seat 0." If seat 0 is not actually participating, the copied answer is not defined.

3. Important Subtopics

Subtopic	Meaning	Why/example	Interview angle
<code>mask</code>	32-bit participation contract.	<code>0xffffffffu</code> is valid only when all 32 live lanes participate.	All named lanes must rendezvous.
<code>value</code>	Per-lane source operand, generally held in a register.	Each lane may hold a different float.	The variable name is the same but values are per thread.
<code>srcLane</code>	Lane index within the logical subgroup.	<code>0</code> broadcasts the subgroup leader.	It is not a global thread ID.
<code>width</code>	Power-of-two logical subgroup size, default 32.	<code>width=8</code> gives four broadcasts.	Subgroups are aligned, not arbitrary.
Return value	Copy of the selected source's operand.	Store leader's value in every lane.	Source remains unchanged.
Valid source	Source must be an active participant for defined data.	Check membership for sparse groups.	Masking destination lanes does not synthesize source values.

4. Real-World Example

A warp processes one database-style batch of 32 records. Lane 0 reads a common scale factor and metadata header. `__shfl_sync` broadcasts them to all lanes, avoiding 32 repeated global loads or a shared-memory staging sequence. If multiple warps are in a block, each warp performs its own broadcast.

5. Diagrams / Mental Models

```

width = 4, srcLane = 1

physical lanes:  0 1 2 3 | 4 5 6 7
logical seats:   0 1 2 3 | 0 1 2 3
selected source:  ^   |   ^

```

Each lane in 0..3 reads lane 1.
 Each lane in 4..7 reads physical lane 5.

6. Common Interview Questions

1. **State the purpose of `__shfl_sync`.** It reads a selected lane's register operand within the same warp. Expected: direct exchange and warp scope. Mistake: calling it shared memory.
2. **What does the first argument mean?** It names the participating lanes required to execute the intrinsic. Mistake: saying it is only the lanes allowed to receive data.
3. **What is the default width?** `warpSize`, normally 32 on NVIDIA GPUs. Mistake: confusing width with block size.
4. **Is `srcLane` a thread index?** It is a lane index relative to the logical subgroup, not a global or block thread ID. Mistake: passing `threadIdx.x` without considering the subgroup.
5. **How do you broadcast from lane 0?** `x = __shfl_sync(mask, x, 0);`. Expected: mask must match participants. Mistake: unconditional full mask in a partial warp.
6. **Can every lane choose a different source?** Yes; `srcLane` is evaluated per calling lane. Mistake: assuming one source is fixed warp-wide.
7. **What happens if source lane is inactive?** The result is undefined. Mistake: expecting the caller's own value or zero.
8. **Does `__shfl_sync` synchronize a block?** No; only named lanes of the warp rendezvous. Mistake: using it for cross-warp shared-memory safety.
9. **How is it different from legacy `__shfl`?** The `_sync` form explicitly supplies a participation mask and is the supported model for modern CUDA architectures. Mistake: omitting divergence concerns.
10. **How do you handle a partial final warp?** Compute a membership mask, for example with `__ballot_sync`, and ensure requested sources exist. Mistake: merely replacing the mask while still reading a missing source.

7. Deep-Dive Questions

1. **Is `__activemask()` always the desired mask?** No. It is a snapshot of currently active lanes and may reflect divergence at that point rather than the logical group intended by the algorithm. Establish group membership deliberately.
2. **Can a mask have noncontiguous bits?** Yes, but `srcLane` still refers to physical/logical lane numbering; bits are not compressed into ranks.
3. **How can a sparse group elect and broadcast a leader?** Choose `leader = __ffs(mask) - 1`, then call `__shfl_sync(mask, value, leader)` from every lane in that group.
4. **Does the intrinsic imply a memory barrier?** It provides the documented synchronization needed for its register exchange, not a general block barrier or universal memory fence.
5. **What if lanes supply different masks?** The program violates the collective's participation requirements and behavior is not reliable. All participating lanes should use the same correct member mask.

8. Comparison Tables

Operation	Source selection	Main use
<code>__shfl_sync</code>	Explicit <code>srcLane</code>	Broadcast, gather
<code>__shfl_down_sync</code>	Current lane + delta	Reduction
<code>__shfl_up_sync</code>	Current lane - delta	Scan
<code>__shfl_xor_sync</code>	Current lane XOR mask	Butterfly/all-reduce
Parameter	Controls	Does not control
<code>mask</code>	Which lanes must participate	Compression/ranking of lanes
<code>srcLane</code>	Whose operand is copied	Which lanes execute
<code>width</code>	Logical aligned subgroup	Arbitrary group membership

9. Common Mistakes

- Using a block thread index as `srcLane`.
- Passing `FULL_MASK` from a path not reached by the full warp.
- Broadcasting from lane 0 when lane 0 is not a member.
- Believing mask bits renumber sparse lanes.
- Using the intrinsic as a block barrier.
- Forgetting `width` must be a supported power of two.

10. Edge Cases / Special Cases

- For a sparse mask, elect a set-bit source rather than assuming lane 0.
- A logical subgroup's `srcLane=0` maps to a different physical lane in every `width` segment.
- Threads that have permanently exited must not remain in the mask.
- The result is per caller, so per-lane source indices implement a permutation or gather.
- Check the CUDA Programming Guide for exact overloads supported by the targeted toolkit and architecture.

11. How to Explain in Interview

"`__shfl_sync` copies a value from a chosen lane's register to every participating caller in the same warp. The mask is a rendezvous contract, `srcLane` is relative to the selected logical width, and the source must itself be active for the result to be defined."

12. Quick Revision Notes

- Signature idea: mask, per-lane value, source lane, optional width.
- Broadcast: fixed source for all lanes.
- Gather/permutation: source can differ per lane.
- Default width: `warpSize`.
- Trap: mask is not lane compaction.
- Trap: inactive source gives undefined data.

13. Practice Tasks

1. Broadcast from the first active lane using `__ffs(mask) - 1`.
2. Use per-lane source indices to reverse eight values.
3. Broadcast a different leader within each group of eight lanes.
4. Write a kernel handling `N=45` without an invalid final-warp source.
5. Explain why `__activemask()` inside divergent paths may produce separate groups.

14. Final Cheat Sheet

Item	Answer
Core definition	Direct same-warp lane-to-lane register read
Key arguments	Participation mask, value, source lane, width
Most asked	Mask semantics and partial-warp correctness
Comparison	Direct source vs relative-source shuffle variants
Interview trap	Sparse mask bits are not renumbered
One-line answer	" <code>__shfl_sync</code> is a masked, warp-local register broadcast or gather."

`__ballot_sync`

1. Overview

`__ballot_sync(mask, predicate)` evaluates a predicate in each participating lane and returns a 32-bit mask whose bit i is 1 when lane i 's predicate is nonzero. Every participating lane receives the same result.

It matters because it converts distributed per-thread decisions into one compact warp-wide value. It is used for active-lane discovery, branch classification, leader election, compaction, match detection, allocation aggregation, and partial-warp-safe collectives. Interviewers ask about it because it connects predicates, lane identity, masks, divergence, and bit operations.

2. Core Idea

Think of 32 voters. Each raises a hand if a condition is true. `__ballot_sync` photographs the row: bit 0 records lane 0, bit 1 lane 1, and so on.

```
unsigned full = 0xffffffffu;
int lane = threadIdx.x & 31;
bool valid = threadIdx.x < n;
unsigned valid_mask = __ballot_sync(full, valid);
int valid_count = __popc(valid_mask);
```

All live lanes first participate in the ballot. The predicate determines output bits, not participation. A lane with `valid == false` still executes the ballot but contributes a zero bit. This distinction is central to correct CUDA code.

3. Important Subtopics

Subtopic	Meaning and importance	Example	Interview angle
Input mask	Lanes that must rendezvous for the vote.	Full mask before divergence.	Participation differs from predicate truth.
Predicate	Per-lane Boolean condition.	<code>index < n, value > 0.</code>	False lanes still participate if in input mask.
Result mask	Bitset of true predicates among participants.	<code>0b00101101.</code>	Bit position equals physical lane ID.
Population count	Number of set bits.	<code>__popc(result).</code>	Count matches/valid lanes.
Leader election	Find first set lane.	<code>__ffs(result) - 1.</code>	Guard the zero-mask case.
Rank/compaction	Count set bits below current lane.	<code>__popc(result & __lanemask_lt())</code> or equivalent lower-bit mask.	Produces unique dense offsets.
Any/all votes	Specialized Boolean collectives.	<code>__any_sync, __all_sync.</code>	Ballot retains per-lane identity; any/all do not.

4. Real-World Example

In a GPU memory allocator, every lane may need space for an output item. A ballot identifies requesting lanes. One elected leader performs a single atomic add for the total request count. Each requester computes its rank among set bits and writes to `base + rank`. This replaces up to 32 atomics with one while preserving unique output positions.

5. Diagrams / Mental Models

```

lane:      7 6 5 4 3 2 1 0
predicate: F T F F T T F T
result:    0 1 0 0 1 1 0 1 = 0x4D

__popc(mask)    -> 4 requesting lanes
__ffs(mask) - 1 -> lane 0 is leader
rank of lane 3  -> popcount(bits below bit 3) = 2

```

6. Common Interview Questions

1. **What does `__ballot_sync` return?** A warp mask with one predicate-result bit per lane. Expected: identical result for participants. Mistake: saying it returns a Boolean.
2. **How is input mask different from output mask?** Input says who participates; output says which participants had true predicates. Mistake: using the predicate to decide whether to call the ballot.
3. **How do you count true lanes?** `__popc(ballot_result)`. Mistake: looping through 32 bits unnecessarily.
4. **How do you find a leader?** `__ffs(result) - 1` finds the least-numbered true lane; first check `result != 0`. Mistake: forgetting `__ffs(0)` returns 0, leading to lane -1.
5. **What is a lane's rank among true lanes?** Popcount of result bits below that lane. Expected: useful for compaction. Mistake: using lane ID as the dense rank.
6. **Can false-predicate lanes call the ballot?** Yes; if named in the input mask, they must call it and contribute zero. Mistake: placing the collective only inside `if (predicate)`.
7. **How is ballot used with a partial warp?** Have all existing lanes vote on logical validity, then use the returned membership mask in later collectives. Mistake: letting out-of-range threads return before the ballot when a full live-warp mask is used.
8. **How does ballot differ from `__any_sync`?** Ballot preserves which lanes were true; `any` only says whether at least one was true. Mistake: using ballot when only one Boolean is needed.
9. **Can ballot communicate across warps?** No. Each warp returns its own 32-bit result. Mistake: treating it as a block vote.
10. **Why is ballot useful before divergence?** It captures the logical group so lanes can use a stable membership mask in later warp collectives. Mistake: recomputing membership from `__activemask()` at arbitrary divergent points.

7. Deep-Dive Questions

1. **How does warp-aggregated atomic allocation work?** Ballot requesters, count them, elect one requester, perform one atomic for the count, broadcast the returned base, and add each requester's rank.
2. **What if the ballot result is zero?** There is no elected lane and no later collective should request a source from that group. Branch around `__ffs`-based leader use.
3. **Why might `FULL_MASK` be valid even when some predicates are false?** Predicate-false lanes still reach the ballot; the full mask describes participation, not truth.
4. **Can ballot results be stored and reused later?** Yes only while the named lanes remain live and reach the later collective consistently. Exits or control-flow changes can invalidate the participation contract.
5. **How can ballot implement partitioning?** Ballot each category predicate, obtain category masks, compute per-category rank with lower bits, and reserve output ranges. Multiple ballots trade instructions for fewer atomics and coherent later work.

8. Comparison Tables

Primitive	Output	Preserves lane identity?	Typical use
<code>__ballot_sync</code>	32-bit mask	Yes	Compaction, membership
<code>__any_sync</code>	Boolean/int	No	Early-exit decision
<code>__all_sync</code>	Boolean/int	No	Warp-wide invariant
<code>__activemask</code>	Current active mask	Yes	Inspect current execution state

Concept	Meaning
Input mask bit = 1	Lane is required to participate
Predicate = true	Lane's output bit becomes 1
Output bit = 1	Participating lane satisfied predicate

9. Common Mistakes

- Calling ballot only from predicate-true lanes while using a wider input mask.
- Confusing the participation mask with the returned vote mask.
- Forgetting the zero-mask case before leader election.
- Treating set-bit order as automatically compressed lane numbering.
- Reusing a stale mask after some named lanes exit.
- Expecting block-wide voting from a warp intrinsic.

10. Edge Cases / Special Cases

- `__ffs(0)` is zero; subtracting one produces an invalid lane.
- A partial logical group may be sparse, so rank must be based on set bits, not lane ID.
- A mask stored before a loop may become invalid if lanes leave the loop permanently at different times.
- The last hardware warp of a block still has only the threads actually created by the block; construct participation from live lanes correctly.
- Ballot output is commonly an `unsigned`; use matching bit operations to avoid signed-shift surprises.

11. How to Explain in Interview

"`__ballot_sync` performs a warp vote: every participating lane evaluates a predicate and receives a bitmask showing which lane predicates were true. The input mask defines participants; the returned mask defines matches. Combining it with popcount and find-first-set enables counting, leader election, and compaction."

12. Quick Revision Notes

- Result bit i corresponds to lane i .
- `__popc(mask)`: member count.
- `__ffs(mask) - 1`: first member, only if mask is nonzero.
- Lower-set-bit popcount: member rank.
- Trap: false predicate does not mean "do not participate."
- Trap: `__activemask` is execution state, not always logical membership.

13. Practice Tasks

1. Count positive numbers in each warp.
2. Elect the first lane whose value exceeds a threshold.
3. Compact qualifying values using ballot, one atomic, and per-lane rank.
4. Produce separate masks for even and odd keys.
5. Handle a zero-result mask without an invalid shuffle source.
6. Trace ballot output and ranks for a hand-written eight-lane predicate pattern.

14. Final Cheat Sheet

Item	Answer
Core definition	Per-lane predicates packed into a warp bitmask
Why it matters	Exposes membership for count, rank, leader, compaction
Most asked	Input vs output mask, <code>__popc</code> , <code>__ffs</code>
Key comparison	Ballot identifies lanes; any/all return only a Boolean
Interview trap	Predicate-false lanes may still be required participants
One-line answer	"Ballot converts 32 distributed decisions into one shared warp mask."

Warp Reductions

1. Overview

A warp reduction combines one value per participating lane into one result using an associative operation such as sum, maximum, minimum, AND, or OR. A tree reduction needs `log2(group_size)` combine stages rather than one lane reading all values sequentially.

Warp reductions matter in statistics, dot products, softmax, histograms, loss computation, attention, and block/grid reductions. Interviewers ask candidates to implement one, reason about masks and partial warps, and distinguish **reduce** (one lane needs the result) from **all-reduce** (every lane needs it).

2. Core Idea

For a full 32-lane sum:

```
__device__ float warp_sum(float x) {
    unsigned mask = 0xffffffffu;
    for (int offset = warpSize / 2; offset > 0; offset /= 2)
        x += __shfl_down_sync(mask, x, offset);
    return x; // lane 0 has the full sum
}
```

Analogy: in a knockout bracket, pairs combine their totals; winners combine again until one total remains.

```
32 values -> 16 pair sums -> 8 -> 4 -> 2 -> 1
offsets      16      8      4      2      1
```

After offset 16, lower lanes add upper partners. After five stages, lane 0 contains the total. Other lanes contain partial sums unless the result is broadcast or an XOR all-reduce is used.

3. Important Subtopics

Subtopic	Meaning and importance	Example	Interview angle
Associativity	Tree regrouping must be mathematically valid.	Sum, min, max.	Floating-point sum is not exactly associative.
Identity value	Neutral element for missing inputs.	0 for sum, -INF for max.	Padding requires the correct identity.
Reduce vs all-reduce	One lane vs every lane receives final result.	Lane 0 writes output vs all lanes normalize.	Do not claim every lane has result after down-shuffle tree.
Full vs partial warp	Number and layout of valid contributors.	Last 13 elements of an array.	Mask alone does not inject identity values.
Block reduction	Warp results are combined across warps.	One shared slot per warp.	Needs shared memory and block synchronization.
Grid reduction	Block results require another kernel, atomics, or cooperative grid sync.	Sum a large array.	Kernel launch is a global synchronization boundary.
Numeric stability	Operation order affects rounding.	Pairwise sum often better than linear sum, but not reproducible across shapes.	Discuss precision and determinism.

4. Real-World Example

In layer normalization, each thread accumulates part of a row. A warp reduction combines partial sums and squared sums. The resulting mean and variance are broadcast to normalize elements. If the row spans several warps, each warp writes one partial to shared memory; the first warp reduces those partials after `__syncthreads()`.

5. Diagrams / Mental Models

```

8-lane sum: [a b c d e f g h]
offset 4:   [a+e b+f c+g d+h ...]
offset 2:   [a+c+e+g b+d+f+h ...]
offset 1:   [a+b+c+d+e+f+g+h ...]
            ^ lane 0 owns final result
  
```

Block-level hierarchy:

```

lane values -> warp reductions -> one partial per warp
              -> shared memory -> barrier
  
```

```
-> first warp reduces partials -> block result
```

6. Common Interview Questions

1. **What is a warp reduction?** A tree combine of per-lane values within one warp. Expected: associative operator and warp scope. Mistake: describing a serial loop.
2. **Why is it $O(\log W)$ stages?** Each stage doubles the number of original inputs represented by a surviving partial. Mistake: calling total hardware work $O(\log W)$; the critical stages are logarithmic, while multiple lanes perform operations.
3. **Why use `__shfl_down_sync`?** It directly supplies partner register values without shared memory. Mistake: claiming it works across warps.
4. **Which lane has the answer?** Usually lane 0 in a down-shuffle tree. Mistake: saying all lanes do.
5. **How do you make it an all-reduce?** Broadcast lane 0 after reduction or use a suitable XOR butterfly. Mistake: reading partial values as final.
6. **How do you handle fewer than 32 inputs?** Use identities for noncontributors and a participation scheme that never requests invalid sources, or use a group collective that supports the group. Mistake: changing only `FULL_MASK` to a short mask.
7. **How do you reduce an entire block?** Reduce per warp, write one partial per warp to shared memory, synchronize the block, then reduce partials with one warp. Mistake: omitting `__syncthreads()`.
8. **How do you reduce an entire grid?** Use atomics, a second kernel, or a cooperative grid launch when supported and correctly sized. Mistake: ordinary blocks synchronizing with each other inside a normal kernel.
9. **Why can floating-point sums differ from CPU sums?** Parallel trees change addition order, and floating-point addition is not associative. Mistake: calling every difference a race.
10. **When is an atomic better?** When there are few block/warp partials or contention is low and simplicity wins. Mistake: assuming atomics are always slow.

7. Deep-Dive Questions

1. **How do you make a block reduction safe when block size is not a multiple of 32?** Guard loads, use identity values, store only real warp leaders, compute the number of warps, then let enough valid lanes in the first warp reduce exactly those partials.
2. **Why can a naive masked down-shuffle fail for 13 lanes?** At some offsets a lane may read from a source not participating. The mask controls synchronization but does not define a neutral return value. Guard each combine or pad live lanes with identities under a safe participation plan.
3. **What affects reduction reproducibility?** Grid/block size, scheduling when atomics are used, compiler transformations, precision, and tree shape. Deterministic algorithms usually cost extra.
4. **How would you reduce a custom type?** Keep fields in supported scalar registers and shuffle/combine each field, or use shared memory/cooperative-group algorithms. The combine operation must define the reduction semantics.
5. **Why use compensated summation?** Kahan/Neumaier-style techniques reduce rounding error for ill-conditioned sums, but add instructions and do not automatically guarantee bitwise reproducibility in a parallel tree.

8. Comparison Tables

Pattern	Final result location	Communication	Typical use
Down-shuffle reduce	Usually lane 0	Shuffle	One warp output
XOR all-reduce	Every lane	Shuffle	Normalize/compare in every lane
Shared-memory block reduce	One block lane	Shuffle + shared memory	Block output
Atomic global accumulation	Global destination	Atomics	Few partials/simple grid result
Two-kernel reduction	Second kernel output	Global memory + launch boundary	Large scalable reduction

Operation	Identity
Sum	0
Product	1
Maximum	Lowest representable value / -INF
Minimum	Highest representable value / +INF
Bitwise OR	0
Bitwise AND	All bits set

9. Common Mistakes

- Assuming every lane contains the final down-shuffle result.
- Using the wrong identity for padded lanes.
- Passing a mask while still reading invalid partners.
- Omitting the block barrier between producing and consuming warp partials.
- Applying a tree to a non-associative operation without defining order.
- Expecting bitwise-identical floating-point results across configurations.
- Building a complicated single-kernel grid reduction when two kernels suffice.

10. Edge Cases / Special Cases

- $N=0$ needs an explicitly defined output, usually the identity.
- NaNs affect `min/max` according to the chosen operation and ordering; define desired NaN policy.
- Overflow can occur even when the mathematical final integer sum fits if intermediate types are too narrow.
- Signed zero and infinities can expose floating-point ordering differences.
- A block may have fewer than one full warp or a non-power-of-two number of warp partials.
- For segmented reductions, lanes must not combine values across segment boundaries.

11. How to Explain in Interview

“A warp reduction combines lane-local values in a logarithmic shuffle tree. A down-shuffle version leaves the final result in lane 0; an all-reduce makes it available to every lane. For partial warps I use valid membership plus identity/partner guards, and for a block I reduce per warp, synchronize through shared memory, then reduce the warp partials.”

12. Quick Revision Notes

- Tree stages for 32 lanes: offsets 16, 8, 4, 2, 1.
- Down-shuffle reduce: lane 0 owns result.
- XOR butterfly: common all-reduce pattern.
- Block: warp partials + shared memory + `__syncthreads()`.
- Grid: atomics, second kernel, or cooperative grid.
- Trap: mask does not provide identity for absent source.
- Trap: floating-point result depends on order.

13. Practice Tasks

1. Implement warp sum and warp maximum for 32 lanes.
2. Extend sum to a 13-element logical group safely.
3. Implement an XOR all-reduce and verify every lane's result.
4. Write a block reduction for block sizes 32, 96, and 100.
5. Build a two-pass grid reduction and compare it with atomic accumulation.
6. Measure FP32 error against an FP64 reference for random and adversarial inputs.

14. Final Cheat Sheet

Item	Answer
Core definition	Associative tree combine across warp lanes
Why it matters	Fast aggregation for sums, extrema, and statistics
Most asked	Shuffle tree, partial warp, block/grid extension
Key comparison	Reduce gives one owner; all-reduce gives every lane the result
Interview trap	A participation mask does not make missing values equal zero
One-line answer	“Warp reduction is a logarithmic lane-combine tree, usually implemented with shuffles.”

Cooperative Groups

1. Overview

CUDA Cooperative Groups is an API for explicitly forming thread groups and applying synchronization or collective operations to those groups. Instead of embedding assumptions such as “this function is called by a whole block” or “these are exactly 32 lanes,” code receives a group object that defines membership and scope.

It matters for safer reusable device functions, warp tiles, block collectives, coalesced active groups, and—under special launch rules—grid-wide synchronization. It is used in reductions, scans, pipelines, multi-stage

kernels, graph work, and persistent kernels. Interviewers ask about it to test synchronization scope, group formation, and the restrictions of `grid.sync()`.

2. Core Idea

Think of a large class split into teams. A barrier for one team should wait only for that team, while a class-wide barrier waits for everyone. A group object tells an algorithm exactly which team it is serving.

```
#include <cooperative_groups.h>
namespace cg = cooperative_groups;

__global__ void kernel(float* x) {
    cg::thread_block block = cg::this_thread_block();
    cg::thread_block_tile<32> tile = cg::tiled_partition<32>(block);

    float v = x[threadIdx.x];
    for (int d = tile.size() / 2; d > 0; d /= 2)
        v += tile.shfl_down(v, d);
    if (tile.thread_rank() == 0) {
        // one result per tile
    }
}
```

Step by step: obtain the current block group; partition it into fixed 32-thread tiles; use group-relative rank and size; perform tile-local shuffles; let the tile leader consume the result. The function no longer needs raw lane-ID arithmetic for basic grouping.

3. Important Subtopics

Subtopic	What it means and why it matters	Example	Interview angle
<code>thread_block</code>	All threads of the current CUDA block.	<code>cg::this_thread_block()</code> .	<code>block.sync()</code> is block scope, similar in purpose to <code>__syncthreads()</code> .
<code>thread_block_tile<N></code>	Compile-time fixed tile partition of a parent group.	<code>tiled_partition<32>(block)</code> .	Useful for warp-sized or smaller collectives.
Runtime tiled group	Partition using a runtime tile size where supported.	Logical subgroups.	Compile-time tiles may enable stronger optimization.
<code>coalesced_group</code>	Currently converged active threads discovered at a point.	<code>cg::coalesced_threads()</code> .	Membership may be dynamic and

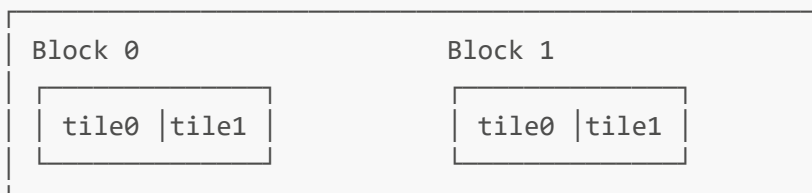
Subtopic	What it means and why it matters	Example	Interview angle
			group-relative ranks are dense.
Group API	<code>size()</code> , <code>thread_rank()</code> , <code>sync()</code> , shuffles, votes, collectives where supported.	Reusable group-based reduction.	Scope follows the group object.
<code>grid_group</code>	All threads in a cooperatively launched grid.	<code>cg::this_grid().sync()</code> .	Requires cooperative launch and residency constraints.
Multi-grid/cluster features	Architecture/runtime- dependent larger or specialized scopes.	Multi-device or thread-block clusters on supported systems.	Do not assume universal availability; check capability and API.

4. Real-World Example

A persistent graph-processing kernel performs several global frontier phases. With a supported cooperative launch, all grid threads process one frontier, write the next frontier, call `grid.sync()`, then continue. This can avoid repeated kernel launches. The tradeoff is strict launch-size/residency requirements; for ordinary applications, separate kernels are often simpler and more scalable because a kernel boundary already provides grid-wide ordering.

5. Diagrams / Mental Models

Grid cooperative group (special cooperative launch)



```

tile.sync() -> only tile members rendezvous
block.sync() -> all block members rendezvous
grid.sync() -> all grid members; cooperative launch required
  
```

6. Common Interview Questions

- 1. What are Cooperative Groups?** CUDA abstractions representing explicit sets of threads with group-scoped synchronization and collectives. Expected: composability and clear scope. Mistake: calling them dynamic CPU thread pools.

2. **Why use a group object?** It makes participation requirements part of a function's interface and avoids hidden block/warp assumptions. Mistake: saying it automatically improves performance.
3. **What is `thread_block`?** The threads in the current block. Expected: `this_thread_block()` and block scope. Mistake: confusing it with the grid.
4. **What is a tiled partition?** A parent group divided into fixed-size subgroups. Expected: group-relative rank and tile-local operations. Mistake: assuming tiles span blocks.
5. **What is `coalesced_group`?** A group of currently converged active lanes discovered at that program point. Mistake: assuming membership remains permanently fixed across arbitrary divergence.
6. **How does `group.sync()` differ by group?** It waits for members of that specific group and applies its documented memory synchronization scope. Mistake: assuming every sync waits for the block.
7. **Can ordinary kernels call `grid.sync()`?** No; the grid must be launched cooperatively on supported hardware/runtime. Mistake: using `this_grid()` as permission by itself.
8. **Why is cooperative grid size restricted?** All blocks participating in a grid barrier must be able to reside concurrently; otherwise waiting blocks could prevent unscheduled blocks from ever reaching the barrier. Mistake: launching an arbitrary huge grid.
9. **When should you prefer multiple kernels?** When global phases are simple, launch overhead is acceptable, or the required grid cannot satisfy cooperative residency. Mistake: forcing a persistent cooperative kernel for elegance.
10. **Can Cooperative Groups replace all CUDA synchronization?** No. Scope, memory ordering, launch constraints, and algorithm needs still determine the correct primitive. Mistake: treating the API as automatic race prevention.

7. Deep-Dive Questions

1. **Why does group-parameterized code improve composability?** A helper taking a group makes the required participants explicit; callers cannot safely invoke a block-wide helper from only one warp without the mismatch becoming visible in the interface.
2. **How is `coalesced_group` different from a ballot mask?** It provides a group abstraction with dense `thread_rank()` for currently converged lanes; a ballot is a physical-lane bitmask that can be stored and manipulated directly.
3. **What causes cooperative launch failure or low occupancy?** Unsupported device capability, too many blocks for concurrent residency, high registers/shared memory per block, or incorrect runtime/API use. Query support and occupancy before launch.
4. **What memory guarantee does `grid.sync()` enable?** It is a grid-wide barrier with the documented visibility needed for participating grid threads across phases; it does not synchronize the host or unrelated kernels/streams.
5. **How would you choose tile size?** Match the algorithm's communication pattern and hardware warp organization, confirm supported collective behavior, and measure register/occupancy effects. A 32-thread tile is natural on NVIDIA but smaller tiles suit subproblems.

8. Comparison Tables

Group	Membership	Formation	Synchronization scope	Main constraint
<code>thread_block</code>	One CUDA block	<code>this_thread_block()</code>	Block	All required block

Group	Membership	Formation	Synchronization scope	Main constraint
				members must reach sync
<code>thread_block_tile<N></code>	Fixed tile within parent	<code>tiled_partition<N></code>	Tile	Tile size/support rules
<code>coalesced_group</code>	Currently converged active lanes	<code>coalesced_threads()</code>	Dynamic group	Membership tied to execution point
<code>grid_group</code>	Whole cooperative grid	<code>this_grid()</code>	Grid	Cooperative launch and residency

Raw intrinsic style**Cooperative Groups style**

Explicit masks and lane IDs	Group object, rank, and size
Very direct hardware mapping	More composable algorithm interface
Easy to hide participation assumptions	Membership is explicit in type/value
Best for tightly tuned primitives	Best for reusable scoped collectives

Grid phase strategy	Advantage	Cost/constraint
Multiple kernels	Simple, scalable global boundary	Kernel-launch overhead, global intermediates
Cooperative <code>grid.sync()</code>	In-kernel global phases	Launch support and all-block residency
Atomic coordination	Flexible incremental progress	Contention and subtle correctness/progress rules

9. Common Mistakes

- Calling a group barrier from only a subset of the group's members.
- Assuming a tile can include threads from different blocks.
- Treating `coalesced_group` membership as fixed forever.
- Launching too many blocks for a cooperative grid barrier.
- Assuming Cooperative Groups makes memory races impossible.
- Using `grid.sync()` without a cooperative launch or capability check.
- Replacing a simple two-kernel algorithm with a constrained persistent kernel without measuring launch overhead.

10. Edge Cases / Special Cases

- Group validity and available operations depend on group type and CUDA version.
- Parent group sizes must satisfy the partitioning rules; do not silently ignore leftover threads.
- Divergence around `sync()` is invalid when some required group members cannot reach it.
- Cooperative occupancy depends on registers, shared memory, block size, and device SM count.
- Multi-device synchronization and thread-block clusters have additional platform and launch requirements.
- A group barrier does not coordinate the CPU, another stream, or unrelated kernels.

11. How to Explain in Interview

“Cooperative Groups makes the participating thread set explicit. I can write an algorithm against a block, warp-sized tile, or current coalesced group and use that group's rank, size, synchronization, and collectives. Grid-wide synchronization is possible only with a supported cooperative launch sized so all blocks can be resident together.”

12. Quick Revision Notes

- `this_thread_block()`: current block group.
- `tiled_partition<N>`: fixed subgroup of a parent.
- `coalesced_threads()`: currently converged active group.
- `this_grid()`: cooperative grid group.
- Group rank is relative to the group, not necessarily physical lane ID.
- Trap: every required group member must reach a barrier.
- Trap: `grid.sync()` needs cooperative launch and residency.

13. Practice Tasks

1. Rewrite a raw shuffle reduction using `thread_block_tile<32>`.
2. Write a helper that takes a group and broadcasts its leader's value.
3. Partition a block of 256 threads into tiles of 32 and emit one sum per tile.
4. Use a `coalesced_group` to assign dense ranks after a divergent predicate.
5. Query cooperative-launch support and compute a legal grid size with occupancy APIs.
6. Compare a two-kernel global reduction with a cooperative-grid version.

14. Final Cheat Sheet

Item	Answer
Core definition	Explicit CUDA thread groups with scoped collectives
Why it matters	Clear participation, reusable device algorithms, wider scopes
Most asked	Block vs tile vs coalesced vs grid group
Key comparison	Raw intrinsics expose masks; groups expose membership and rank
Interview trap	<code>grid.sync()</code> is not legal in an arbitrary normal launch
One-line answer	“Cooperative Groups makes synchronization scope and participants explicit.”

Combined Interview Map

Need	Best starting primitive	Critical correctness question
Broadcast one lane's register	<code>__shfl_sync</code>	Is the source lane participating?
Know which lanes match	<code>__ballot_sync</code>	Did all lanes in the input mask call it?
Sum one warp	<code>__shfl_down_sync</code> tree	Are missing partners handled with valid identities/guards?
Result in every lane	XOR all-reduce or reduce + broadcast	Does every lane need the final value?
Communicate across warps	Shared memory + block synchronization	Did all block participants reach the barrier?
Reusable scoped collective	Cooperative Groups	Does the group exactly match the callers?
Synchronize an entire grid	Cooperative <code>grid_group</code> or kernel boundary	Is cooperative launch legal and resident, or are two kernels simpler?

Final Placement Checklist

- State that a warp is normally 32 NVIDIA CUDA threads executing in SIMT style.
- Distinguish the **participation mask** from a predicate/result mask.
- Never claim an inactive shuffle source returns zero.
- Distinguish reduce from all-reduce.
- Extend warp reduction to block reduction with shared memory and a block barrier.
- Mention floating-point non-associativity when discussing sums.
- Explain that Cooperative Groups makes scope explicit rather than magically removing synchronization rules.
- State the cooperative-launch and residency requirement for `grid.sync()`.
- In an interview, lead with correctness and scope; discuss performance only after the participating threads are well defined.